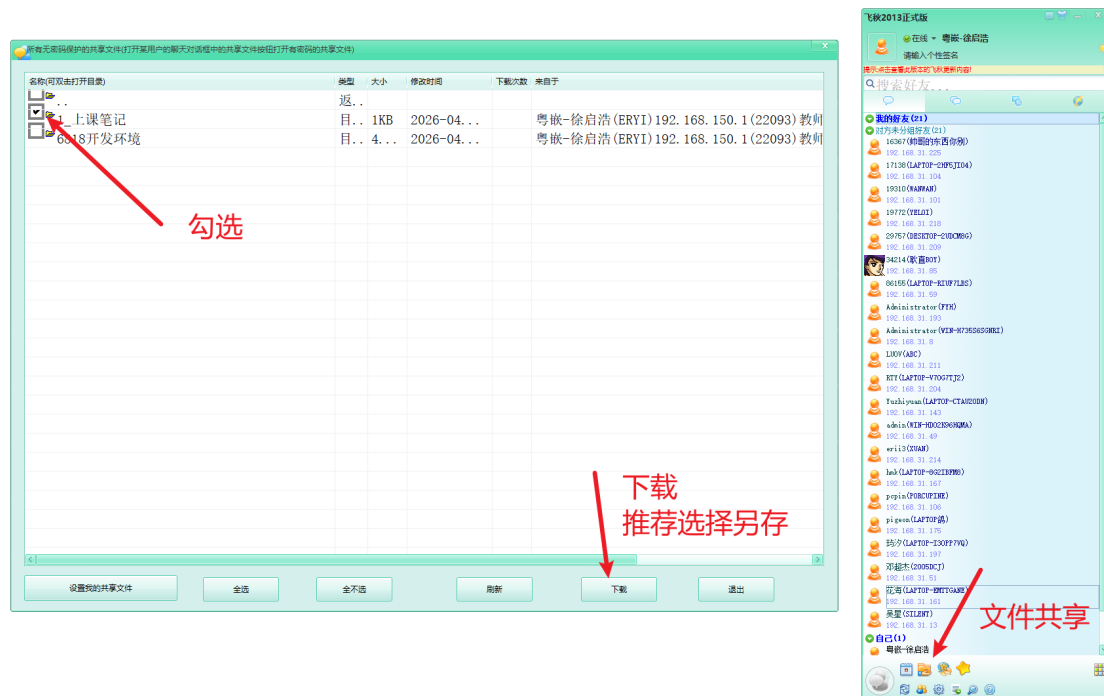


零、个人介绍

我是来自广州粤嵌通信科技股份有限公司湖南分公司的徐启浩

一、文件下载



二、项目介绍

我们这几天会在GEC6818开发板上基于C语言实现一个语音识别项目。

三、Ubuntu程序开发流程

3.1 编写程序

```
1 #include <stdio.h>
2
3 int main()
4 {
5     printf("hello world\n");
6
7     return 0;
8 }
```

3.2 编译程序

代码需要放在共享文件夹

编译: 把代码编译成机器可以识别的语言

编译器: gcc GNU编译器集合

在我们的共享文件夹下, 找到我们写好的代码文件, 之后进行编译

```
1  常用命令格式：
2      gcc  代码文件  -o  可执行文件的名称
3  意义：
4      把某个/某些代码文件编译成一个可以执行的程序文件
5
6  如： gcc 01_hello.c -o hello
7      对01_hello.c 进行编译，生成了一个可执行文件hello
8
9  注意：
10     如果不加-o给可执行文件命名，则可执行文件默认命名为： a.out
```

3.3 执行程序

我们在Ubuntu中使用终端进入到共享文件夹，找到我们编译好的程序文件，并执行

终端: 是用户和系统之间的桥梁，它会解析用户的输入，从而做出对应的操作

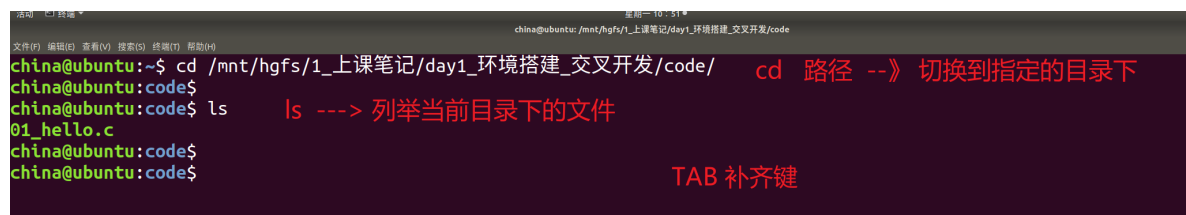
Ubuntu桌面空白处右击 ---》打开终端

```
1  命令格式：
2      ./可执行文件
3  意义：
4      执行当前目录下的某个文件
5
6  如：
7      ./hello
8      执行当前目录下hello这个文件
```

4.4 虚拟机操作演示

step1: 进入到共享文件夹中找到程序文件

cd /mnt/hgfs/xxx ---> xxx是你共享文件夹的名字



The screenshot shows a terminal window with the following commands and output:

```
china@ubuntu:~$ cd /mnt/hgfs/1_上课笔记/day1_环境搭建_交叉开发/code/
china@ubuntu:code$ ls
01_hello.c
```

Annotations in red text:

- `cd 路径 --》 切换到指定的目录下` (next to the cd command)
- `ls ---> 列举当前目录下的文件` (next to the ls command)
- `TAB 补齐键` (next to the empty line after ls)

step2: 编译程序



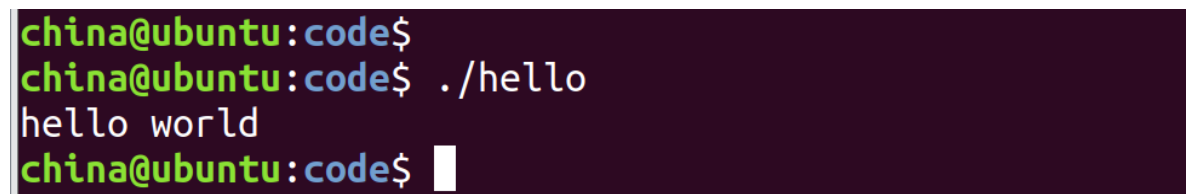
The screenshot shows a terminal window with the following commands and output:

```
china@ubuntu:code$ ls
01_hello.c
china@ubuntu:code$ gcc 01_hello.c -o hello
china@ubuntu:code$ ls
01_hello.c hello
```

Annotations in red text:

- `这里只有一个01_hello.c` (next to the output of the first ls command)
- `编译生成可执行文件hello` (next to the gcc command)
- `多了一个文件hello` (next to the output of the second ls command)

step3: 执行程序



The screenshot shows a terminal window with the following commands and output:

```
china@ubuntu:code$ ./hello
hello world
china@ubuntu:code$
```

四、文件IO

Linux下一切皆文件(设计逻辑、理念)

操作系统把操作文件的底层实现封装起来了，提供了一些接口给我们使用

如果想要访问文件，就需要提供一个 **进程文件表项的下标 ---》文件描述符**，一个被打开的文件，都可以通过这个文件描述符来引用。

在Linux下，同一个进程中，唯一表示一个打开的文件，所有对文件的操作都需要使用到它。

4.1 打开文件

```
1  NAME
2      open, creat - 用来 打开和创建 一个 文件或设备
3
4  SYNOPSIS 总览
5      #include <sys/types.h>
6      #include <sys/stat.h>
7      #include <fcntl.h>
8
9      功能：打开或者创建一个文件
10     int open(const char *pathname, int flags);
11     pathname: 需要打开或者创建的文件的的路径
12     flag: 标志位
13         O_RDONLY    只读打开
14         O_WRONLY    只写打开
15         O_RDWR      读写打开
16         必选标志位，什么方式打开文件，必须三选一
17     返回值：
18         成功返回一个文件描述符，后续对文件的操作都需要使用到这个文件描述符
19         失败返回-1，同时errno被设置
20         错误码可以使用函数perror进行解析
```

4.2 关闭文件

```
1  NAME 名字
2      close - 关闭一个文件描述符
3
4  SYNOPSIS 总览
5      #include <unistd.h>
6
7      功能：关闭一个文件
8     int close(int fd);
9     参数：需要关闭文件对应的文件描述符
```

```
1  #include <sys/types.h>
2  #include <sys/stat.h>
3  #include <fcntl.h>
4  #include <unistd.h>
5  #include <stdio.h>
6  #include <stdlib.h>
7
8
9  int main()
10 {
```

```

11 //只读方式打开文件
12 int fd = open("1.txt", O_RDONLY);
13 if(fd == -1)
14 {
15     perror("open 1.txt error");
16     exit(1);
17 }
18 printf("open 1.txt success\n");
19
20 //关闭文件
21 close(fd);
22
23 return 0;
24 }

```

4.3 文件写入

```

1 NAME
2     write -在一个文件描述符上执行写操作
3
4 概述
5     #include <unistd.h>
6
7 功能: write 向文件描述符 fd 所引用的文件中写入 从 buf 开始的缓冲区中 count 字节的数据
8 ssize_t write(int fd, const void *buf, size_t count);
9     fd: 需要进行写入数据的那个文件对应的文件描述符
10    buf: 指向需要写入的数据对应的存储空间
11    count: 需要写入的数据大小, 单位是字节
12 返回值:
13    成功返回实际写入的字节数
14    失败返回-1, 同时error被设置
15
16 示例:
17 #include <sys/types.h>
18 #include <sys/stat.h>
19 #include <fcntl.h>
20 #include <unistd.h>
21 #include <stdio.h>
22 #include <stdlib.h>
23
24
25 int main()
26 {
27     //只写方式打开文件
28     int fd = open("1.txt", O_WRONLY);
29     if(fd == -1)
30     {
31         perror("open 1.txt error");
32         exit(1);
33     }
34     printf("open 1.txt success\n");
35
36     //往文件中写入数据
37     char buf[10] = {"hello"};
38     int ret = write(fd, buf, 5);
39     if(ret == -1)
40     {

```

```

41     perror("write error");
42     exit(1);
43 }
44 printf("write %d bytes datas\n", ret);
45
46 //关闭文件
47 close(fd);
48
49 return 0;
50 }

```

4.4 文件读取

```

1  NAME
2      read - 在文件描述符上执行读操作
3
4  概述
5      #include <unistd.h>
6
7  功能：read() 从文件描述符 fd 中读取 count 字节的数据并放入从 buf 开始的缓冲区中。
8  ssize_t read(int fd, void *buf, size_t count);
9      fd: 需要进行读取操作的文件对应的文件描述符
10     buf: 指向的空间用来存储从文件中读取到的内容
11     count: 需要读取多少个字节的数据
12 返回值：
13     成功返回实际读取到的字节数(<= count)
14     失败返回-1, 同时errno 被设置

```

```

1  #include <sys/types.h>
2  #include <sys/stat.h>
3  #include <fcntl.h>
4  #include <unistd.h>
5  #include <stdio.h>
6  #include <stdlib.h>
7
8  int main()
9  {
10     //只读方式打开文件
11     int fd = open("1.txt", O_RDONLY);
12     if(fd == -1)
13     {
14         perror("open 1.txt error");
15         exit(1);
16     }
17     printf("open 1.txt success\n");
18
19     //从文件中读取数据
20     char buf[10] = {0};
21     int ret = read(fd, buf, sizeof(buf)); //sizeof(buf)buf这个数组的大小
22     if(ret == -1)
23     {
24         perror("read error");
25         exit(1);
26     }
27     printf("read %d bytes datas: %s\n", ret, buf);
28

```

```
29 //关闭文件
30 close(fd);
31
32 return 0;
33 }
```

五、交叉开发

一般来说，研发嵌入式产品，从产品成本及功能的专用性角度出发考虑。

嵌入式产品一般只有程序的运行环境，而没有程序的编译开发环境。

所以，我们一般只有在通用电脑上用各种编译开发软件把程序调试好后，再下载到开发板或者相关的产品上去运行。**这个过程称之为叫交叉开发**

编写程序: Windows

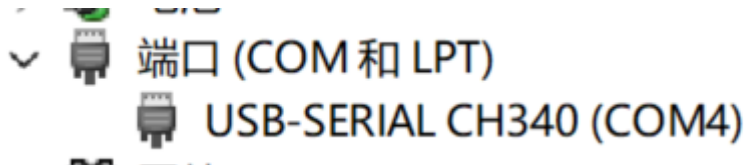
编译程序: Ubuntu

执行程序: GEC6818开发板

5.1 开发板的连接

step1: 开发板与电脑连接，并开机

step2: 打开设备管理器(搜索栏搜索设备管理器)，获取端口号



如图，我这里识别到了端口号**CH340 (COM4)**

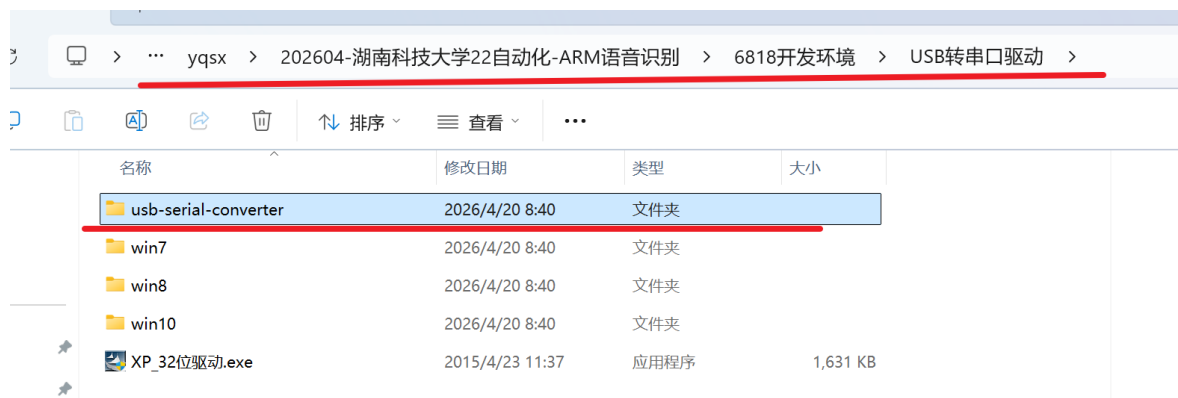
端口号的格式: **CH340 COMx(x是一个数字)**

问题:开发板与电脑连接好了，但是设备管理器中没有端口号

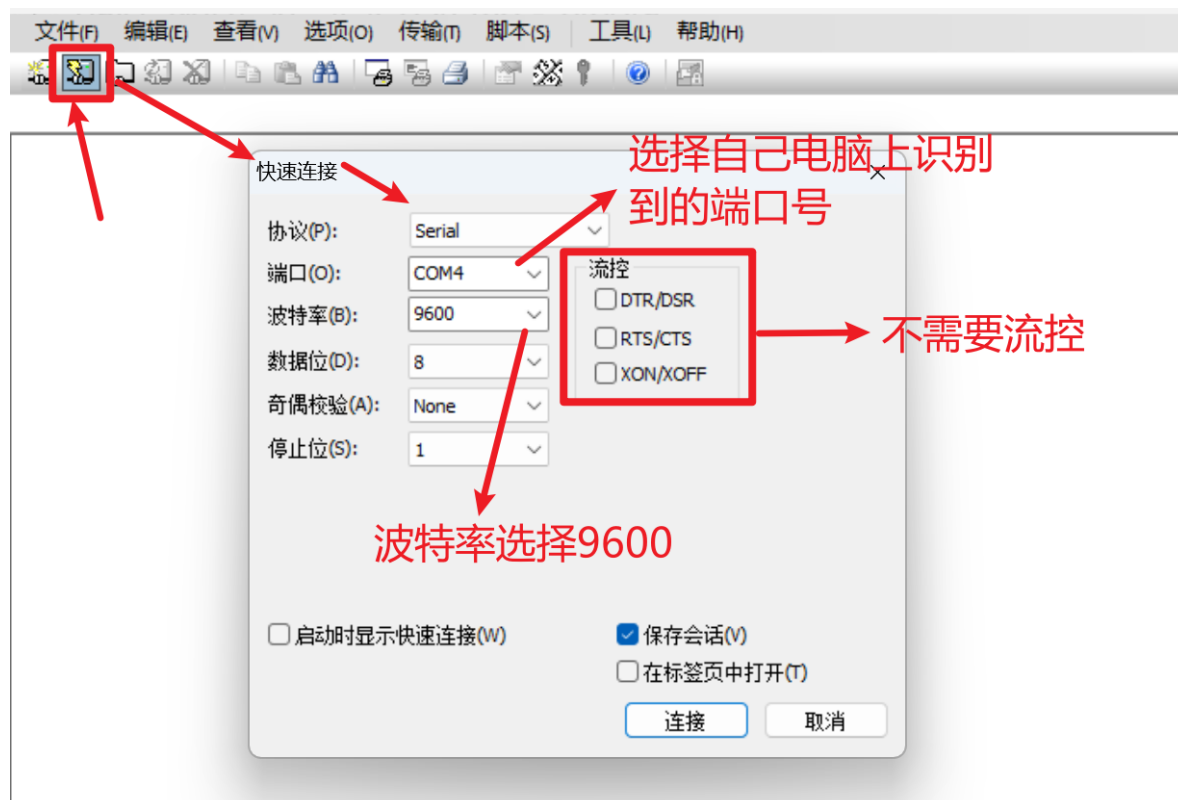
解决: 安装对应的驱动程序即可



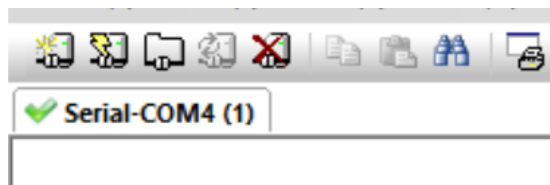
如果上述驱动程序安装之后，仍然没有端口号，则安装下面的驱动



step3: 打开SecureCRT这个工具，对开发板进行连接

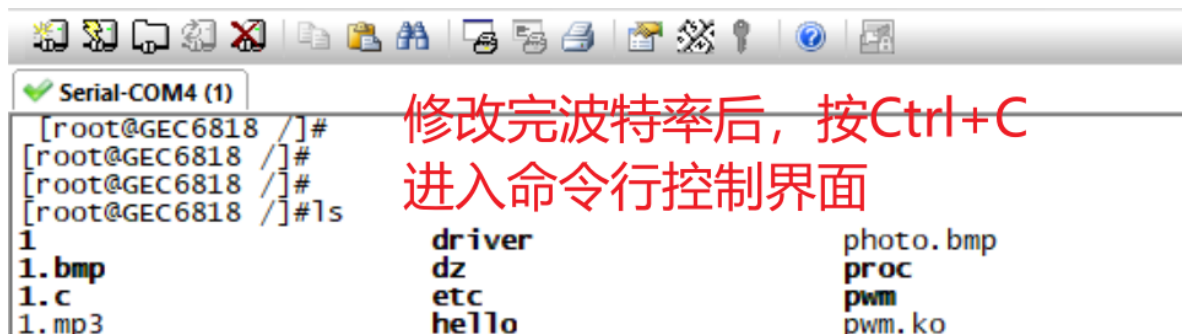
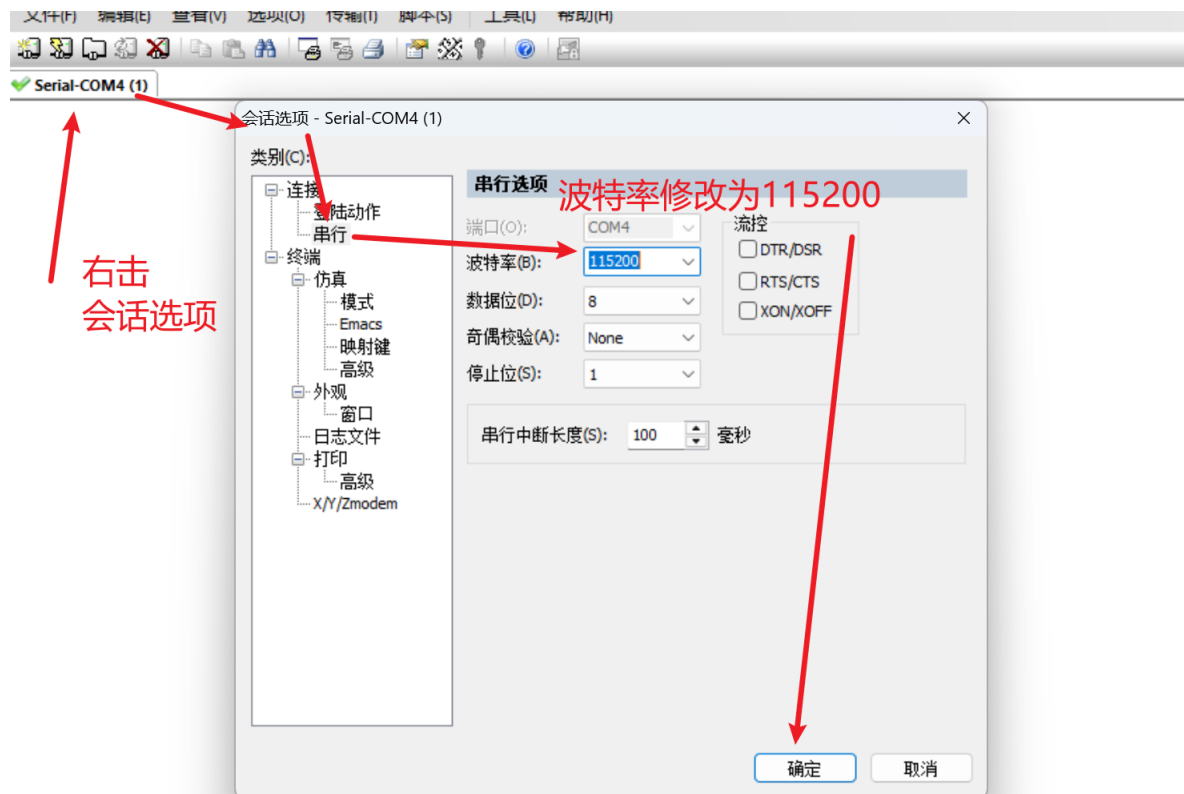


点击连接

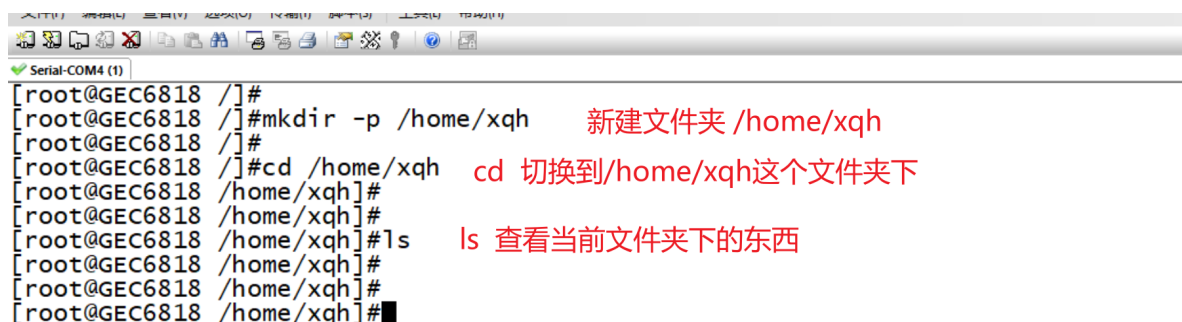


连接成功

连接成功后，修改波特率为115200



step4: 建立家目录，后续使用到的图片、程序等等全部放到家目录下



5.2 交叉开发流程

step1: 编写程序

```
1 #include <stdio.h>
2
3 int main()
4 {
5     printf("hello world\n");
6
7     return 0;
8 }
```

step2: 编译程序

由于ARM处理器和Inter处理器其设计结构有本质的区别

所以在ARM开发板上运行的程序，则必须使用专门的编译器来编译

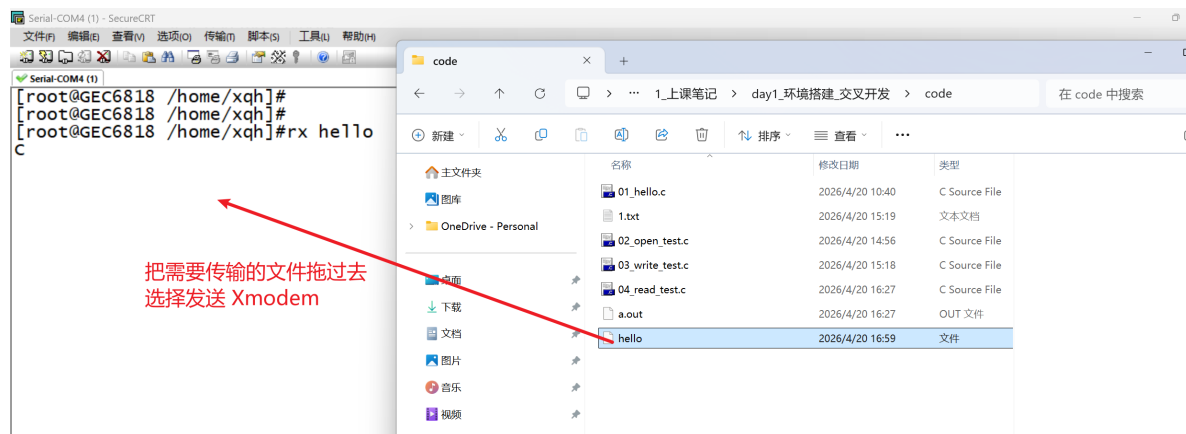
arm-linux-gcc

```
1 常用命令格式：
2     arm-linux-gcc 代码文件 -o 可执行文件的名称
3 意义：
4     把某个/某些代码文件编译成一个可以执行的程序文件
5
6 如： arm-linux-gcc 01_hello.c -o hello
7     对01_hello.c 进行编译，生成了一个可执行文件hello
8
9 注意：
10    如果不加-o给可执行文件命名，则可执行文件默认命名为： a.out
```

```
china@ubuntu:code$ arm-linux-gcc 01_hello.c -o hello
```

step3: 执行程序

```
1 将可执行文件传输到开发板
2
3 命令格式：
4     rx 文件名 回车
```



传输成功后，执行程序，无法执行

```
[root@GEC6818 /home/xqh]#  
[root@GEC6818 /home/xqh]#ls  
hello  
[root@GEC6818 /home/xqh]#  
[root@GEC6818 /home/xqh]#./hello  
-:/bin/sh: ./hello: Permission denied  
[root@GEC6818 /home/xqh]#  
[root@GEC6818 /home/xqh]#
```

- 1 我们需要给文件添加可执行权限，
- 2 `chmod +x 文件路径`
- 3
- 4 如：
- 5 `chmod +x ./hello`
- 6
- 7 添加完权限之后，就可以执行程序了
- 8 `./hello`

```
[root@GEC6818 /home/xqh]#chmod +x ./hello  
[root@GEC6818 /home/xqh]#  
[root@GEC6818 /home/xqh]#./hello  
hello world  
[root@GEC6818 /home/xqh]#  
[root@GEC6818 /home/xqh]#
```